

Python Programming Assignment (I)

Object-Oriented Analysis Systems | Choose one project

Course	Submission	Permitted tools
Basic Python through classes	GitHub repository link and commit hash	Python standard library

Choice: Complete either Option A or Option B. Both options have equivalent scope.

1. Purpose and learning outcomes

This assignment asks you to design a small but complete Python application using object-oriented programming. You will translate a written problem into classes, functions, validation rules and readable reports.

- define classes with clear responsibilities;
- create and use objects, constructors, attributes and methods;
- demonstrate encapsulation and composition;
- use inheritance and method overriding where they improve the design;
- decompose calculations into reusable functions;
- process data stored in lists and dictionaries; and
- document, test and communicate a software design through GitHub.

2. Design freedom

You are not required to use prescribed class or function names. You must decide how to represent the problem and explain your decisions in the README. Your implementation must, however, satisfy the common functional and object-oriented requirements below.

Suggestion: Focus on justified design, correct behaviour and readable code. Creating many classes does not automatically produce a better design.

3. Common requirements

Whichever option you choose, your solution must include:

- at least four meaningful classes, including one clear example of composition;
- at least one private or protected-style attribute controlled through methods or a property;
- inheritance and method overriding used meaningfully, or a written explanation of why composition is preferable for your design;
- at least one class method or static method with a justified purpose;
- at least four standalone functions for calculations, validation or presentation;

- lists and dictionaries as part of the application data;
- validation of missing, impossible or poor-quality values;
- a structured result returned as a dictionary;
- a readable console report; and
- at least five scenarios, including normal, unusual and invalid-data cases.

You may use Python's standard-library modules, including math and statistics. Do not use pandas, NumPy, scikit-learn, databases, external APIs, graphical interfaces or machine-learning models.

OPTION A | Smart Fitness Session Analyzer

Background

A fitness centre receives simulated measurements from wearable devices used during training sessions. It needs a Python program that organizes participants and exercise sessions, validates measurements, compares measurements with personal reference values, classifies session intensity and describes recovery after activity.

Data supplied to the program

A typical observation may contain:

```
{ "timestamp": 2, "heart_rate": 118, "skin_response": 2.5,
  "temperature": 32.9, "activity_level": 0.72,
  "signal_quality": 0.93 }
```

Required behaviour

- Represent a participant and the participant's reference measurements.
- Represent individual observation windows and group them into a session.
- Accept valid observations and reject or flag invalid observations.
- Calculate suitable summaries, including average, minimum and maximum values.
- Compare heart rate and other measurements with the participant's reference values.
- Classify a session as resting, moderate activity, high activity, recovering or insufficient data.
- Detect recovery by examining whether heart rate and activity decline near the end of a session.
- Report how many observations were usable and explain the classification.

Minimum scenarios

- resting session
- moderate activity
- high activity
- activity followed by recovery
- poor-quality or invalid sensor data

OPTION B | Podcast Voice and Recording Analyzer

Background

A podcast company uses a separate audio tool to extract numerical characteristics from short speech windows. The original audio and spoken content are not available to your program. Your task is to organize speakers and recording sessions, validate the extracted measurements, compare a session with the speaker's usual recording profile, and generate an interpretable quality and speaking-style report.

Data supplied to the program

A typical observation may contain:

```
{"timestamp": 2, "speech_present": True, "pitch": 148.0,  
  "energy": 0.43, "speech_rate": 126, "pause_ratio": 0.19,  
  "background_noise": 0.11, "signal_quality": 0.92}
```

Required behaviour

- Represent a speaker and the speaker's usual acoustic profile.
- Represent acoustic observation windows and group them into a recording session.
- Validate pitch, energy, speech rate, ratios, noise and signal-quality values.
- Exclude or flag windows without usable speech.
- Calculate suitable summaries for each recording feature.
- Compare the recording with the speaker's usual profile.
- Describe the session as consistent, energetic, deliberate, temporarily varied, noise affected or insufficient data.
- Evaluate overall recording quality and explain the result.

Minimum scenarios

- consistent speaking style
- energetic delivery
- slow or deliberate delivery
- high background noise
- missing speech or poor recording quality

4. GitHub repository and submission

Create a public repository or a private repository accessible to the instructor. Submit both the repository URL and the commit hash representing the final assessed version.

Required repository contents

```
your-repository/  
|-- README.md  
|-- main.py  
|-- sample_data.py  
|-- tests.py  
`-- requirements.txt
```

You may use a different structure. If you split the solution into additional modules, explain the structure in the README. The program must run from the repository root without changing paths or source code.

README requirements

- project title and selected option;
- student name and student number;
- short description of the application;
- class design and the responsibility of each class;
- where composition, encapsulation, inheritance and overriding are demonstrated;
- assumptions and classification rules;
- exact installation and running instructions;
- example output; and
- known limitations.

Minimum running instructions

```
git clone https://github.com/USERNAME/REPOSITORY.git  
cd REPOSITORY  
python3 main.py
```

If your operating system uses the command `python` rather than `python3`, state this alternative in the README. If no third-party packages are used, `requirements.txt` may be empty or may state that the project uses only the Python standard library.